Choix des technos : 2
Projet 1 2
Création du manifest : 2
Projet 2 4
Test 7
Schéma de l'architecture : 8

Choix des technos :

Pour réaliser ces deux projets, les solutions que j'ai choisi sont tout d'abord GitHub Desktop pour le versioning, Multipass pour gérer mes machines virtuelles, Visual Studio Code comme IDE pour accéder à ma VM en SSH, Docker pour builder et push mes applications sur le dockerhub, et minikube pour l'utilisation de kubernetes.

Projet 1

Le lien du repo Git : https://github.com/SoniKast/projet_4SFARCH

Choix des applications de la VM :

sonikast1/django-adminlte

sonikast1/ecommerce-flask-stripe-1.0.11

sonikast1/django-star-admin

Création du manifest :

Pour le premier projet, j'ai décidé de commencer avec des applications que j'avais déjà optimisé auparavant afin de me simplifier la tâche et de gagner du temps. J'ai donc tout déployé dans un seul fichier manifest avec une optique « infra as code ». La structure est ainsi :

- Le Namespace « prod-b-05-03-2026 » est initialisé avant tous les autres composants, car tous les autres vont être initialisés dans le namespace
    - Le Service « django-adminlte-svc-b » avec le type ClusterIP
    - Le Deployment « django-adminlte-b-05-03-2026 »
    - Le Service « django-star-admin-svc-b » avec le type NodePort
    - Le Deployment « django-star-admin-b-05-03-2026 »
    - Le Service « ecommerce-flask-stripe-svc-b » avec le type LoadBalancer
    - Le Deployment « ecommerce-flask-stripe-b-05-03-2026 »

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: prod-b-05-03-2026
  labels:
    name: prod-b-05-03-2026
---
apiVersion: v1
kind: Service
metadata:
  name: django-adminlte-svc-b
  labels:
    app: django-adminlte-svc-b
  namespace: prod-b-05-03-2026
spec:
  type: ClusterIP
  selector:
    app: django-adminlte-b-05-03-2026
  ports:
  - port: 5005
    protocol: TCP
    targetPort: 5005
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: django-adminlte-b-05-03-2026
  labels:
    app: django-adminlte-b-05-03-2026
  namespace: prod-b-05-03-2026
spec:
  replicas: 3
  selector:
    matchLabels:
      app: django-adminlte-b-05-03-2026
  template:
    metadata:
      labels:
        app: django-adminlte-b-05-03-2026
    spec:
      containers:
      - name: django-adminlte-b-05-03-2026
        imagePullPolicy: IfNotPresent
        image: sonikast1/django-adminlte:latest
        ports:
        - containerPort: 5005
---
```

Les 3 applications se trouvent dans le même manifest. Ils peuvent êtres crées avec la commande « k create –f p1-balla-05-03-2026.yml », et accédées dans le navigateur en faisant un portforward. « k port-forward —address 0.0.0.0 –n prod-b-05-03-2026 services/ ‘’nomduservice’’ 5005:5005 »

Lancement des services et deployments :

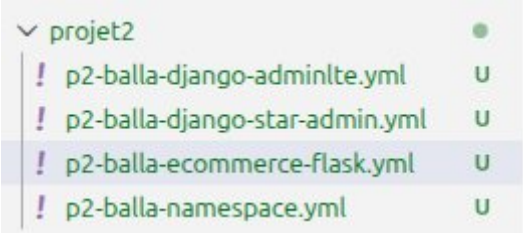
Après la création des services, j'ai utilisé la commande « k get all –n prod-b-05-03-2026 » afin de voir si tous les pods, services, deployments et replicas fonctinnaient correctement.

```
ubuntu@mydocker:~/projet_4SFARCH$ k get all -n prod-b-05-03-2026
NAME                                                    READY   STATUS              RESTARTS   AGE
pod/django-adminlte-b-05-03-2026-69ff4478d9-2nssr       0/1     ContainerCreating   0          24s
pod/django-adminlte-b-05-03-2026-69ff4478d9-jb5jl       0/1     ContainerCreating   0          24s
pod/django-adminlte-b-05-03-2026-69ff4478d9-st5kf       0/1     ContainerCreating   0          24s
pod/django-star-admin-b-05-03-2026-6897c96894-f5ltf     0/1     ContainerCreating   0          23s
pod/django-star-admin-b-05-03-2026-6897c96894-lf7bq     0/1     ContainerCreating   0          23s
pod/django-star-admin-b-05-03-2026-6897c96894-tphzs     0/1     ContainerCreating   0          23s
pod/ecommerce-flask-stripe-b-05-03-2026-755466679f-52f8k   0/1   ContainerCreating   0          20s
pod/ecommerce-flask-stripe-b-05-03-2026-755466679f-pznkf   0/1   ImagePullBackOff    0          21s
pod/ecommerce-flask-stripe-b-05-03-2026-755466679f-xwgk7   0/1   ContainerCreating   0          20s

NAME                                    TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/django-adminlte-svc-b           ClusterIP      10.108.125.104   <none>        5005/TCP         27s
service/django-star-admin-svc-b         NodePort       10.102.238.37    <none>        5005:32625/TCP   27s
service/ecommerce-flask-stripe-svc-b    LoadBalancer   10.97.137.108    <pending>     5005:32197/TCP   26s

NAME                                                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/django-adminlte-b-05-03-2026            0/3     3            0           28s
deployment.apps/django-star-admin-b-05-03-2026          0/3     3            0           28s
deployment.apps/ecommerce-flask-stripe-b-05-03-2026     0/3     3            0           26s

NAME                                                       DESIRED   CURRENT   READY   AGE
replicaset.apps/django-adminlte-b-05-03-2026-69ff4478d9    3         3         0       28s
replicaset.apps/django-star-admin-b-05-03-2026-6897c96894  3         3         0       27s
replicaset.apps/ecommerce-flask-stripe-b-05-03-2026-755466679f   3   3         0       26s
```

Ensuite, j'ai testé tous les deployments en port-forward, et tout était OK.

Projet 2

Pour le projet 2, j'ai repris les mêmes applications, mais cette fois-ci je les ai configurés dans des fichiers différents :



Chaque manifest, à l'exception de ecommerce-flask, contient un service et un deployment :

- Django AdminLTE est en ClusterIP
- Django Star Admin est en NodePort
- Ecommerce Flask est en LoadBalancer (que j'ai ensuite remplacé par Rocket Ecommerce)

Pour l'application Ecommerce Flask, j'ai rajouté en plus deux composants en plus : ConfigMap pour les variables d'environnements et un Secret pour gérer les clés d'API, juste avant le deployment et après le service, pour transmettre les clés d'API Stripe. Afin de suivre les normes pour les Secret, les deux clés d'API Stripe dans le manifest sont convertis en Base64.

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: ecommerce-flask-stripe-cm-b
  namespace: prodv2-b-05-03-2026
data:
  DEMO_MODE: "True"
  DEBUG: "True"
---
apiVersion: v1
kind: Secret
metadata:
  name: ecommerce-flask-stripe-secret-b
  namespace: prodv2-b-05-03-2026
type: Opaque
data:
  STRIPE_PUBLISHABLE_KEY: cGtfdGVzdF81MV(
  STRIPE_SECRET_KEY: c2tfdGVzdF81MVQ2dmZ)
```

Je n'ai pas pu tester l'application ecommerce-flask avec stripe correctement, même si tout était bien configuré, donc j'ai du changer d'application pour partir sur « rocket-ecommerce »

J'ai optimisé le Dockerfile et le docker-compose de l'application Rocket Ecommerce :

```dockerfile
FROM python:3.11.5-alpine3.18

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt

COPY . .

RUN apk add --update nodejs npm
RUN npm i && npm run build

#python manage.py collectstatic --no-input & python manage.py makemigrations & python manage.py migrate
CMD ["gunicorn", "--config", "gunicorn-cfg.py", "core.wsgi"]
```

```yaml
nginx:
  container_name: nginx
  restart: always
  image: "nginx:alpine-slim"
  ports:
    - "5085:5085"
  volumes:
    - ./nginx:/etc/nginx/conf.d
  networks:
    - web_network
  depends_on:
    - rocket-django
    - myflaskdb
  healthcheck:
    test: "curl --silent --fail http://localhost:5085 > /dev/null || exit 1"
```

Puis j'ai build et push l'application sur mon docker hub :

```
ubuntu@mydocker:~/priv-rocket-ecommerce-main$ docker build -t sonikast1/rocket-ecommerce-main:latest .
[+] Building 235.6s (12/12) FINISHED                                                                                docker:default
 => [internal] load build definition from Dockerfile                                                                          5.2s
 => => transferring dockerfile: 465B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.11.5-alpine3.18                                                   8.3s
 => [auth] library/python:pull token for registry-1.docker.io                                                                 0.0s
 => [internal] load .dockerignore                                                                                             5.6s
 => => transferring context: 2B                                                                                               0.0s
 => [1/6] FROM docker.io/library/python:3.11.5-alpine3.18@sha256:cd311c6a0164f34a7edbf364e05258b07d66d3f7bc155139dcb9bef88a186ded 31.7s
 => => resolve docker.io/library/python:3.11.5-alpine3.18@sha256:cd311c6a0164f34a7edbf364e05258b07d66d3f7bc155139dcb9bef88a186ded 5.8s
 => => sha256:79094e0cad96b6b0e4d5651c38e6f0242021d8ca446b7ae86771c908f09f5c66 3.11MB / 3.11MB                                1.6s
 => => sha256:a2c7770dd0b99cea7bc658a1b8532610f1fc0ef61557c683ed779e6c554442b2 242B / 242B                                    1.8s
 => => sha256:083c1679885df5e7d7c72833ab00be841b5429f67b791ee88a54bc20cd45b537 12.47MB / 12.47MB                              2.7s
 => => sha256:9875af95546db78168a6761b7fa205ed1cd0c153cd89356c1512e551c12b2d5c 622.29kB / 622.29kB                            2.3s
 => => sha256:96526aa774ef0126ad0fe9e9a95764c5fc37f409ab9e97021e7b4775d82bf6fa 3.40MB / 3.40MB                                2.1s
 => => extracting sha256:96526aa774ef0126ad0fe9e9a95764c5fc37f409ab9e97021e7b4775d82bf6fa                                     0.7s
 => => extracting sha256:9875af95546db78168a6761b7fa205ed1cd0c153cd89356c1512e551c12b2d5c                                     1.5s
 => => extracting sha256:083c1679885df5e7d7c72833ab00be841b5429f67b791ee88a54bc20cd45b537                                     1.8s
 => => extracting sha256:a2c7770dd0b99cea7bc658a1b8532610f1fc0ef61557c683ed779e6c554442b2                                     0.5s
 => => extracting sha256:79094e0cad96b6b0e4d5651c38e6f0242021d8ca446b7ae86771c908f09f5c66                                     0.7s
 => [internal] load build context                                                                                            12.3s
 => => transferring context: 17.13MB                                                                                          0.1s
 => [2/6] COPY requirements.txt .                                                                                             3.1s
 => [3/6] RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt                                    72.7s
 => [4/6] COPY . .                                                                                                           13.0s
 => [5/6] RUN apk add --update nodejs npm                                                                                    17.3s
 => [6/6] RUN npm i && npm run build                                                                                         21.3s
 => exporting to image                                                                                                       42.8s
 => => exporting layers                                                                                                      23.9s
 => => exporting manifest sha256:a4bace29235fe402d3b5a79ab238ef58666b8a7634c6903ae4f035293719096d                              1.3s
 => => exporting config sha256:61deb854be95c7f1ac72496ed57cfeafdabc7fc1e302dc94edaab6ac46d7fd5c                                1.0s
 => => exporting attestation manifest sha256:003c22358a6a16c7d831467fcc438ba6583e543b3b4393eea2f1a10c43bedcd4                  2.4s
 => => exporting manifest list sha256:615c13743100b491aeb6d279d14b1ce5765b192bfa7febde67b3df7256ad882a                         0.7s
 => => naming to docker.io/sonikast1/rocket-ecommerce-main:latest                                                             0.2s
 => => unpacking to docker.io/sonikast1/rocket-ecommerce-main:latest                                                         11.9s
ubuntu@mydocker:~/priv-rocket-ecommerce-main$ docker push sonikast1/rocket-ecommerce-main:latest
The push refers to repository [docker.io/sonikast1/rocket-ecommerce-main]
96526aa774ef: Pushed
a2c7770dd0b9: Pushed
e28db28d35e6: Pushed
083c1679885d: Pushed
9875af95546d: Pushed
cd9903dca146: Pushed
6b080a742589: Pushed
f67129bc3865: Pushed
79094e0cad96: Pushed
5488e4e162ab: Pushed
a62c1f757f29: Pushed
latest: digest: sha256:615c13743100b491aeb6d279d14b1ce5765b192bfa7febde67b3df7256ad882a size: 856
```

En premier temps j'avais ajouté une base de données MySQL, mais j'ai eu des problèmes pour la mise en place des services, donc j'ai retiré cette partie la du Docker-compose, j'ai re-push l'application, et tout fonctionnait.

J'ai donc en suite supprimé le namespace pour reprendre :

```
ubuntu@mydocker:~/projet_4SFARCH$ k delete namespaces prodv2-b-05-03-2026
namespace "prodv2-b-05-03-2026" deleted
```

J'ai relancé la commande « k create –f ./projet2 » pour remettre le projet en place. Ensuite j'ai exécuté la commande « k get all –n prodv2-b-05-03-2026 » pour vérifier si tout est fonctionnel dans le namespace:

```
ubuntu@mydocker:~/projet_4SFARCH$ k get all -n prodv2-b-05-03-2026
NAME                                                    READY   STATUS    RESTARTS   AGE
pod/django-adminlte-b-05-03-2026-69ff4478d9-47mlc       1/1     Running   0          15m
pod/django-adminlte-b-05-03-2026-69ff4478d9-568gn       1/1     Running   0          15m
pod/django-adminlte-b-05-03-2026-69ff4478d9-lwnvf       1/1     Running   0          15m
pod/django-star-admin-b-05-03-2026-6897c96894-5f2jp     1/1     Running   0          15m
pod/django-star-admin-b-05-03-2026-6897c96894-w9wbt     1/1     Running   0          15m
pod/django-star-admin-b-05-03-2026-6897c96894-wfbz4     1/1     Running   0          15m
pod/rocket-ecommerce-main-b-05-03-2026-6c89df8bfc-6szpd 1/1     Running   0          62s
pod/rocket-ecommerce-main-b-05-03-2026-6c89df8bfc-p2tqg 1/1     Running   0          39s
pod/rocket-ecommerce-main-b-05-03-2026-6c89df8bfc-v7cpd 1/1     Running   0          112s

NAME                                 TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/django-adminlte-svc-b        ClusterIP      10.105.119.2     <none>        5005/TCP         15m
service/django-star-admin-svc-b      NodePort       10.106.5.242     <none>        5005:31749/TCP   15m
service/rocket-ecommerce-main-svc-b  LoadBalancer   10.105.134.254   <pending>     5005:30548/TCP   15m

NAME                                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/django-adminlte-b-05-03-2026        3/3     3            3           15m
deployment.apps/django-star-admin-b-05-03-2026      3/3     3            3           15m
deployment.apps/rocket-ecommerce-main-b-05-03-2026  3/3     3            3           15m

NAME                                                   DESIRED   CURRENT   READY   AGE
replicaset.apps/django-adminlte-b-05-03-2026-69ff4478d9   3         3         3       15m
replicaset.apps/django-star-admin-b-05-03-2026-6897c96894 3         3         3       15m
replicaset.apps/rocket-ecommerce-main-b-05-03-2026-6c89df8bfc 3     3         3       113s
replicaset.apps/rocket-ecommerce-main-b-05-03-2026-95c5c8b79  0     0         0       15m
```

Test

Puis, j'ai réalisé un portforward : « k port-forward —address 0.0.0.0 –n prodv2-b-05-03-2026 service/rocket-ecommerce-main-svc-b 5005:5005 » sur le port 5005, et en y accédant dans le navigateur, j'ai testé un paiement et tout fonctionne.

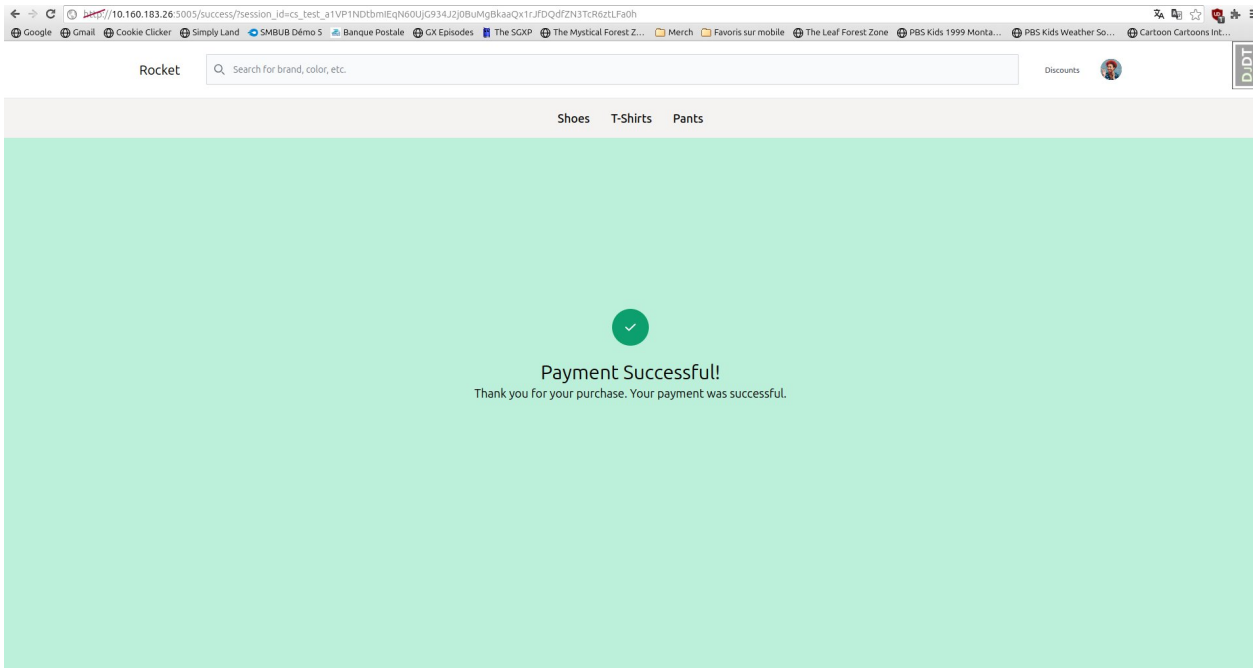
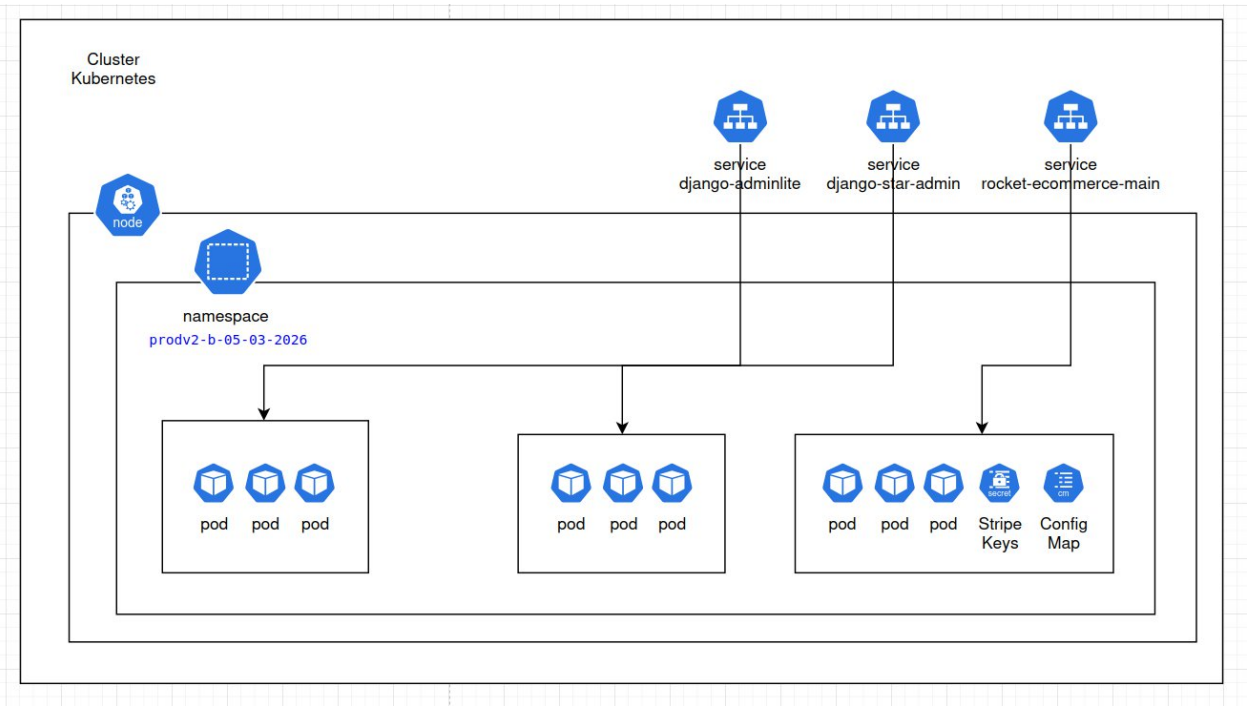


Schéma de l'architecture :

Voici un schéma de l'architecture :



Expliquer le choix concernant les frameworks et technologies utilisées avec les avantages et inconvénients

Pour les frameworks et technologies utilisées, j'ai choisi MiniKube et Kubernetes car ils permettent une orchestration des containeurs, et Kubernetes a un système de ReplicaSets, qui m'a permis d'implémenter 3 réplicas par application dans ce projet. Comme inconvénients, ils ont une courbe d'apprentissage assez élevée et une consommation de ressources importante pour un simple environnement local.

Pour les machines virtuelles, j'ai utilisé MultiPass pour gérer mes machines virtuelles, c'était la solution la plus simple pour moi car j'utilise Linux Mint et non Windows. Son système de yaml est très utile pour la réplication. L'inconvénient de MultiPass est que son utilisation sur Windows est moins bonne que sur Linux.

J'ai utilisé Docker et Docker Hub pour builder les images et les stocker, l'avantage est la portabilité totale des applications. L'inconvénient est que la gestion des images peut devenir complexe sans une stratégie de nettoyage régulière.

Les applications que j’ai utilisé pour les deux projets sont des applications en Django (AdminLTE & Star Admin) et en Flask (Ecommerce Stripe et Rocket Ecommerce) l’avantage avec ces frameworks sont qu’ils sont relativement simple et légers à mettre en place.